

Activity 5: Objective-C++ Adapter

Course: AI Vibe Coding for iOS Mobile Apps Development (TGS-2023037545)

Mapping: K3, A3

Estimated time: 85 minutes

Objective

Expose the portable domain core to Swift through a narrow Objective-C-compatible API.

Deliverable

Bridge header, .mm adapter and round-trip test

Tools

Xcode, Objective-C++, XCTest

Included files

- `starter/BudgetBridge.h`
- `starter/BudgetBridge.mm`
- `solution/BudgetBridge.h`
- `solution/BudgetPolicy.hpp`
- `solution/BudgetBridge.mm`
- `solution/bridge-test.swift`

Before you begin

- Work only with the supplied mock data. Do not add credentials, personal data or signing files.
- Keep the portable C++ core free of Swift, Objective-C and Apple framework dependencies.
- Record the initial state before editing so that verification shows a meaningful change.
- If using an AI coding assistant, treat its output as a proposal and inspect the resulting diff.

Procedure

1. Run `cd activities/05-objective-cpp-adapter` and inspect the starter API with `sed -n '1,200p' starter/BudgetBridge.h` and `sed -n '1,200p' starter/BudgetBridge.mm`.
2. Create a working copy with `cp -R starter workspace`. Place the Activity 2 `BudgetPolicy.hpp` on the Objective-C++ target's header search path; do not import it from `BudgetBridge.h`.
3. In the Xcode app target, add `workspace/BudgetBridge.h` and `workspace/BudgetBridge.mm`. Confirm the implementation suffix is `.mm`; a `.m` file cannot compile C++ constructs.
4. Keep the public header limited to Foundation-compatible declarations, fixed-width integers, Objective-C objects and `NSError \*\*`. Run `rg -n 'std::|#include <vector>|#include <string>' workspace/BudgetBridge.h` and expect no matches.

5. In `BudgetBridge.mm`, include `BudgetPolicy.hpp` and add a private `std::unique_ptr<BudgetPolicy>` owner. Initialise it once in `-init`.
6. Translate `limit`, `spent` and `draft` from `int64_t` into `Money` value objects, call `evaluate`, then copy accepted and remaining-cents values into `BudgetResultDTO`.
7. Map a rejected domain decision to the stable error domain `BudgetDomain` and code `1001`. Do not expose a raw C++ exception or pointer to Swift.
8. Add the supplied `solution/bridge-test.swift` pattern to an XCTest target. Ensure the app module is imported with `@testable import` when appropriate.
9. Run the focused test in Xcode, or use `xcodebuild test -scheme BudgetBuddy -destination 'platform=iOS Simulator,name=iPhone 17 Pro' -only-testing:BudgetBuddyTests/BudgetBridgeTests` after substituting the actual scheme and available simulator.
10. Expect the exact input `10_000`, spent `2_500` and draft `1_250` to return remaining cents `6_250`; add a second test that verifies rejection maps to error code `1001`.
11. If Swift cannot see the API, check target membership, the bridging header or module import, and that no C++ type appears in the public header before changing the domain core.
12. Retain the public header, `.mm` adapter, passing test result and a short ownership note showing Swift retains the adapter while the adapter owns the C++ service through RAIL.

Verification

Swift can call the bridge; cents, identity and error codes survive the round trip exactly.

Capture the command output, test result, matrix or screenshot named by the deliverable. A statement such as “it works” is not sufficient evidence.

Troubleshooting

If Swift cannot import the header, check target membership and confirm C++ includes appear only in `.mm`.

Check the earliest failing layer first: portable C++ domain test, then Objective-C++ contract, then Swift state and finally the visible interface.

Cleanup

Clean derived build data only if needed; retain tests and mapping table.

Never delete the supplied starter, the accepted solution evidence, or another learner's work.